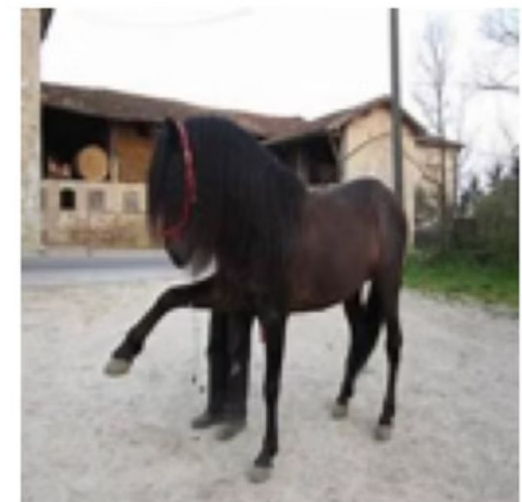
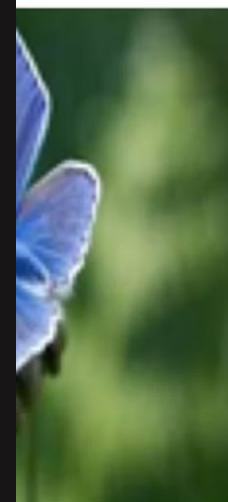
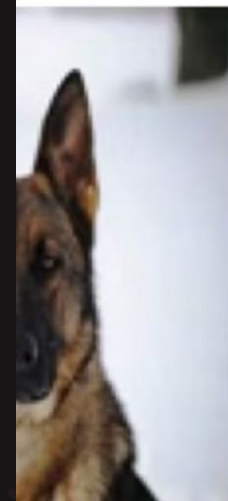
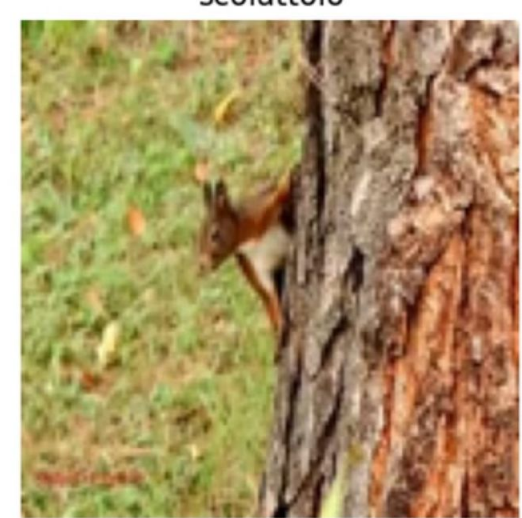


Animal Image Classification Using CNN

Introduction to Applied AI – Final Project
Anish Reddy Nukala



Problem Definition

Manual image classification is time-consuming and impossible to scale. AI offers a faster, automated alternative by learning visual patterns directly from data.

Project Goal

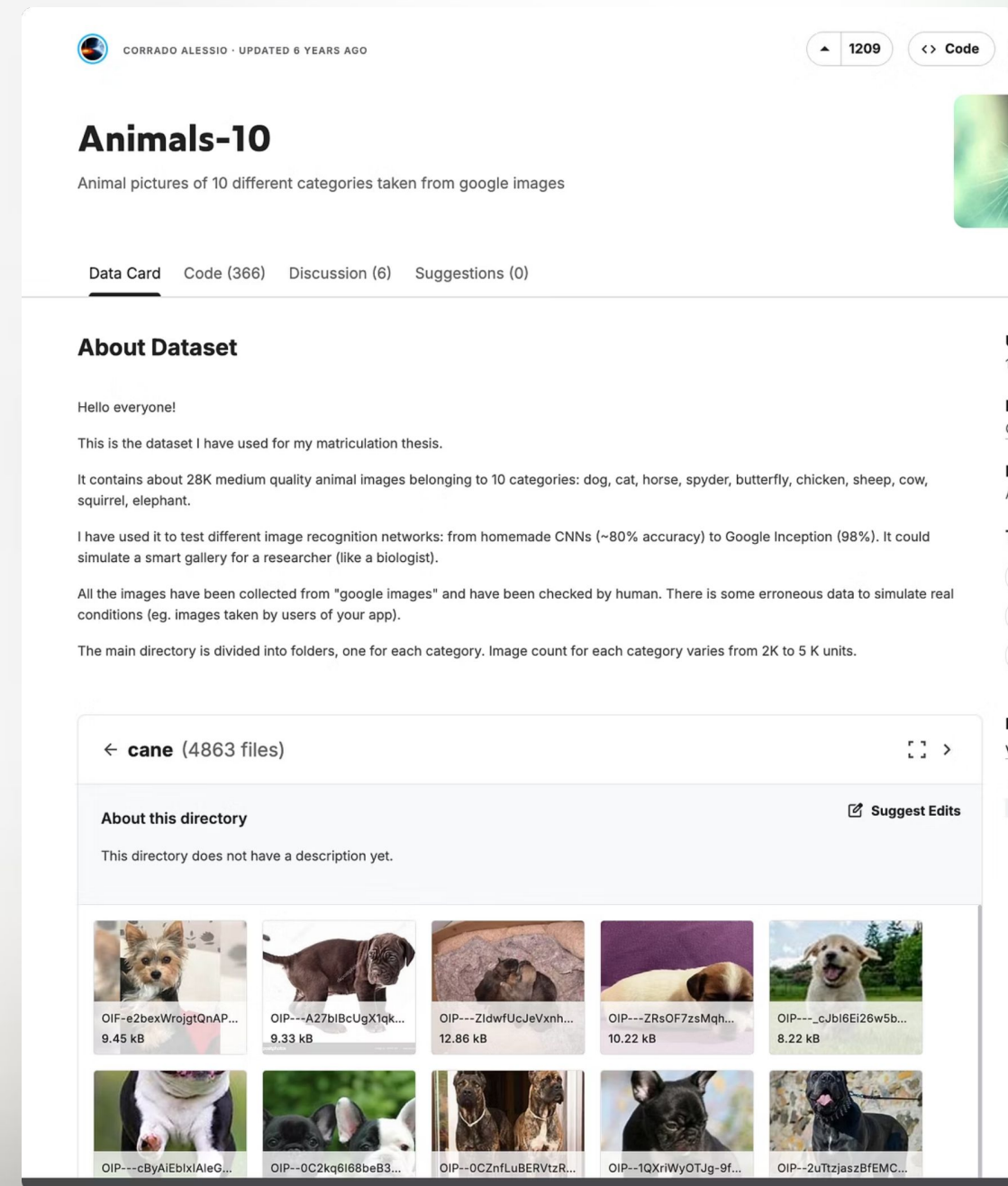
Automatically classify animal images into **Dog**, **Cat**, or **Horse** using a trained CNN model.

What CNNs Learn

Shapes, textures, colors, ears, faces, and body features — patterns humans use too, but at scale.

Why It Matters

Scalable, consistent, and fast — a CNN can process thousands of images in seconds without fatigue.



Dataset Description

ANIMALS-10 · KAGGLE · ALESSIOCORRAD099

Check Dataset Folders and Class Names

```
[ ] ▶ class_translation = {
    "cane": "Dog",
    "gatto": "Cat",
    "cavallo": "Horse",
    "ragno": "Spider",
    "farfalla": "Butterfly",
    "gallina": "Chicken",
    "pecora": "Sheep",
    "mucca": "Cow",
    "scoiattolo": "Squirrel",
    "elefante": "Elephant"
}

for folder in os.listdir(dataset_path):
    folder_path = os.path.join(dataset_path, folder)
    if os.path.isdir(folder_path):
        print(f"{folder} ({class_translation.get(folder, folder)}): {len(os.listdir(folder_path))} images")
```

... cavallo (Horse): 2623 images
pecora (Sheep): 1820 images
elefante (Elephant): 1446 images
gatto (Cat): 1668 images
scoiattolo (Squirrel): 1862 images
gallina (Chicken): 3098 images
ragno (Spider): 4821 images
mucca (Cow): 1866 images
cane (Dog): 4863 images
farfalla (Butterfly): 2112 images

The dataset used is **Animals-10** from Kaggle. It contains images for the following 10 classes:

- **Dog (cane):** 4863 images
- **Cat (gatto):** 1668 images
- **Horse (cavallo):** 2623 images
- **Spider (ragno):** 4821 images
- **Butterfly (farfalla):** 2112 images
- **Chicken (gallina):** 3098 images
- **Sheep (pecora):** 1820 images
- **Cow (mucca):** 1866 images
- **Squirrel (scoiattolo):** 1862 images
- **Elephant (elefante):** 1446 images

For training and validation, the `data_loader` function is configured to use only 'Dog', 'Cat', and 'Horse' classes.

Dataset Details

The **Animals-10** dataset contains 10 animal classes. This project uses **3 selected classes** for faster training in Google Colab.

- Images organized in folders by class

Downloaded in Google Colab via `kagglehub`

Resized to **96×96 pixels**

Split: **80% training / 20% validation**



Cane

Dog



Gatto

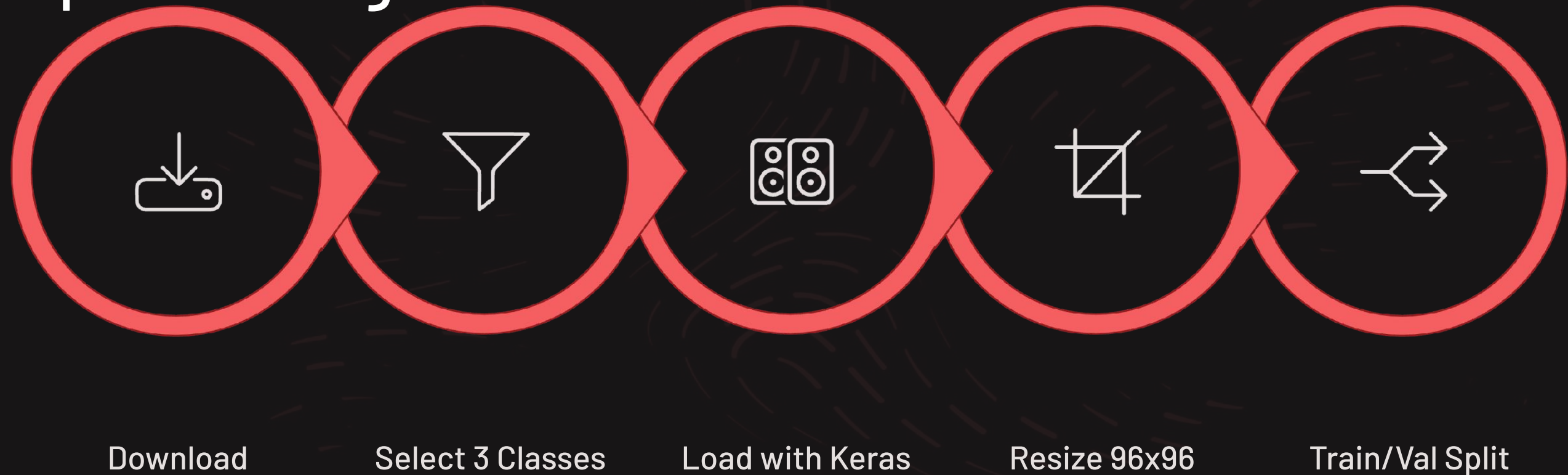
Cat



Cavallo

Horse

Methodology & Preprocessing



Data augmentation — including random flip, rotation, zoom, translation, contrast, and brightness adjustments — was applied to improve generalization and reduce overfitting on the small dataset.

CNN Model Architecture

Display Model Summary

model = build_model(
 input_shape=(96, 96, 3),
 num_classes=len(class_names)
)

model.summary()

... Model: "sequential"

Layer (type)	Output Shape	Param #
data_augmentation (Sequential)	(None, 96, 96, 3)	0
rescaling (Rescaling)	(None, 96, 96, 3)	0
conv2d (Conv2D)	(None, 94, 94, 32)	896
max_pooling2d (MaxPooling2D)	(None, 47, 47, 32)	0
conv2d_1 (Conv2D)	(None, 45, 45, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 22, 22, 64)	0
conv2d_2 (Conv2D)	(None, 20, 20, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 128)	0
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 128)	1,638,528
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 1,732,163 (6.61 MB)
Trainable params: 1,732,163 (6.61 MB)
Non-trainable params: 0 (0.00 B)

Compile and Train the Model

Architecture Highlights

Input shape: **96×96×3** (RGB)

Three **Conv2D + MaxPooling** blocks (32 → 64 → 128 filters)

Dropout layer to prevent overfitting

Softmax output for 3-class probability

Training Configuration

Optimizer: Adam

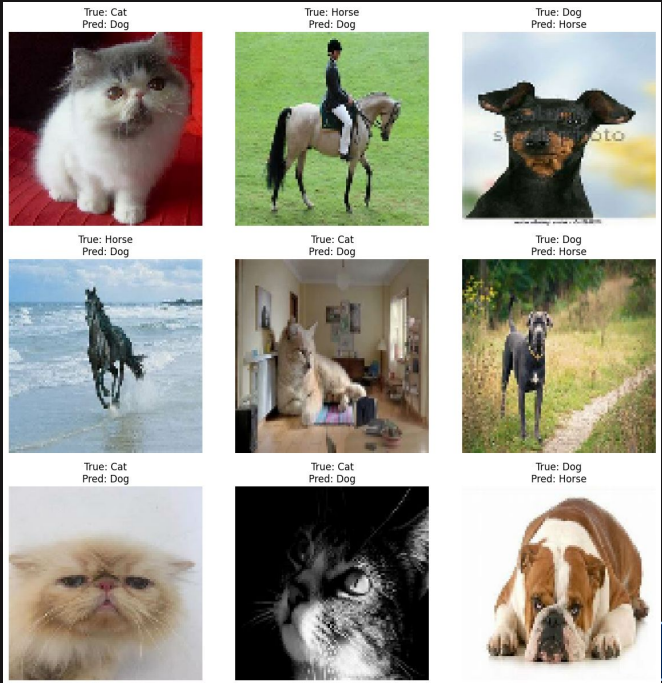
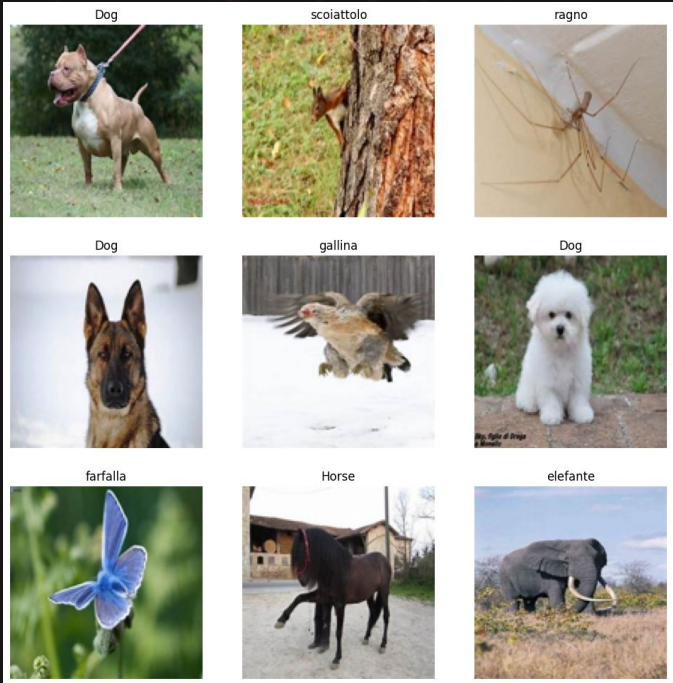
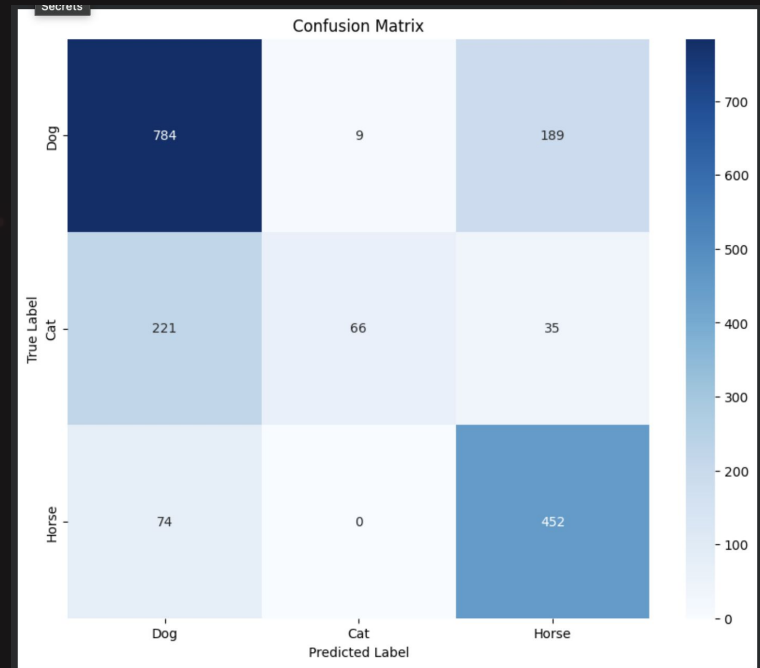
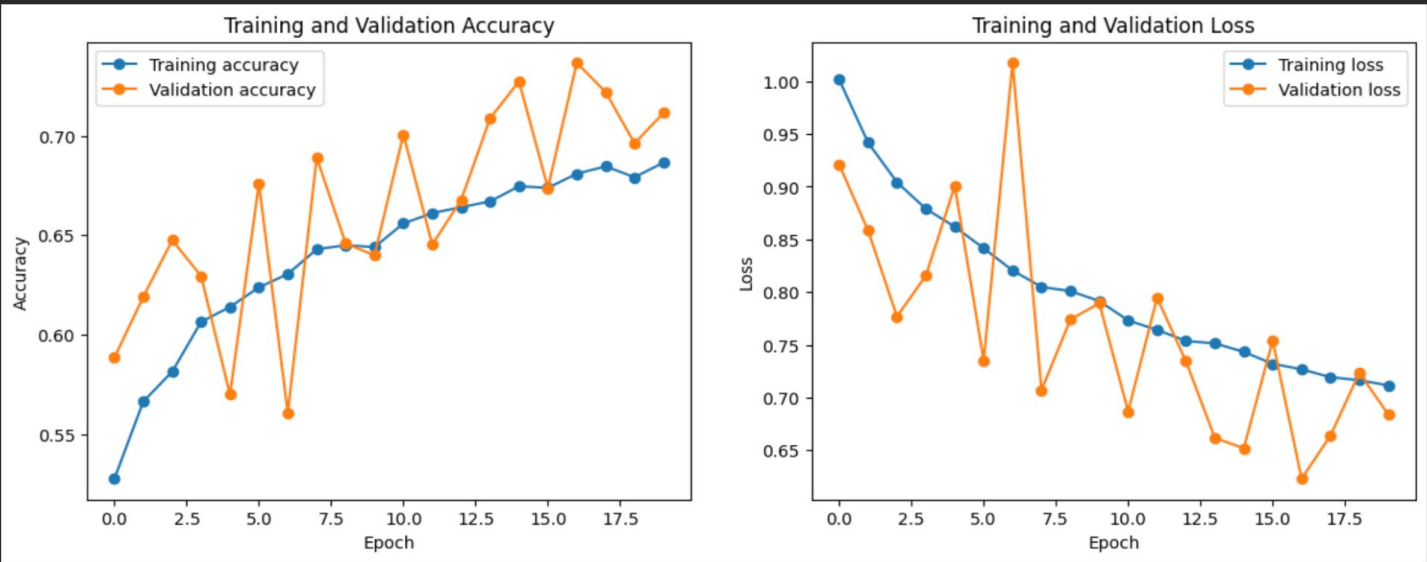
Loss: Sparse Categorical Crossentropy

Metric: Accuracy

Results & Evaluation

The metrics below will be populated after model training. Accuracy reflects overall correctness; precision, recall, and F1-score reveal per-class performance in detail.

Epoch 1/20	48s 27ms/step	accuracy: 0.5277	loss: 1.0015	val_accuracy: 0.5885	val_loss: 0.9211
Epoch 2/20	45s 45s				
45s 45s	15s 16ms/step	accuracy: 0.5665	loss: 0.9417	val_accuracy: 0.6191	val_loss: 0.8583
Epoch 3/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.5815	loss: 0.9042	val_accuracy: 0.6475	val_loss: 0.7769
Epoch 4/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6065	loss: 0.8790	val_accuracy: 0.6295	val_loss: 0.8156
Epoch 5/20	45s 45s				
45s 45s	16s 16ms/step	accuracy: 0.6140	loss: 0.8620	val_accuracy: 0.5699	val_loss: 0.9008
Epoch 6/20	45s 45s				
45s 45s	16s 17ms/step	accuracy: 0.6238	loss: 0.8416	val_accuracy: 0.6760	val_loss: 0.7352
Epoch 7/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6305	loss: 0.8205	val_accuracy: 0.5607	val_loss: 1.0174
Epoch 8/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6430	loss: 0.8050	val_accuracy: 0.6891	val_loss: 0.7072
Epoch 9/20	45s 45s				
45s 45s	16s 17ms/step	accuracy: 0.6450	loss: 0.8010	val_accuracy: 0.6459	val_loss: 0.7740
Epoch 10/20	45s 45s				
45s 45s	20s 15ms/step	accuracy: 0.6440	loss: 0.7916	val_accuracy: 0.6399	val_loss: 0.7896
Epoch 11/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6559	loss: 0.7732	val_accuracy: 0.7005	val_loss: 0.6870
Epoch 12/20	45s 45s				
45s 45s	21s 16ms/step	accuracy: 0.6611	loss: 0.7642	val_accuracy: 0.6454	val_loss: 0.7948
Epoch 13/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6641	loss: 0.7537	val_accuracy: 0.6678	val_loss: 0.7344
Epoch 14/20	45s 45s				
45s 45s	16s 17ms/step	accuracy: 0.6670	loss: 0.7514	val_accuracy: 0.7007	val_loss: 0.6618
Epoch 15/20	45s 45s				
45s 45s	16s 18ms/step	accuracy: 0.6746	loss: 0.7434	val_accuracy: 0.7273	val_loss: 0.6520
Epoch 16/20	45s 45s				
45s 45s	15s 16ms/step	accuracy: 0.6738	loss: 0.7321	val_accuracy: 0.6738	val_loss: 0.7542
Epoch 17/20	45s 45s				
45s 45s	15s 15ms/step	accuracy: 0.6809	loss: 0.7269	val_accuracy: 0.7366	val_loss: 0.6231
Epoch 18/20	45s 45s				
45s 45s	16s 16ms/step	accuracy: 0.6846	loss: 0.7194	val_accuracy: 0.7219	val_loss: 0.6640
Epoch 19/20	45s 45s				
45s 45s	16s 16ms/step	accuracy: 0.6791	loss: 0.7166	val_accuracy: 0.6962	val_loss: 0.7235
Epoch 20/20	45s 45s				
45s 45s	15s 17ms/step	accuracy: 0.6865	loss: 0.7116	val_accuracy: 0.7115	val_loss: 0.6841



Conclusion & Future

Work

Key Takeaways

- CNN successfully learned to distinguish Dog, Cat, and Horse images
- 3-class scope enabled fast, manageable training in Colab
- Data augmentation improved generalization & reduced overfitting
- Demonstrates real-world applicability of computer

vision

Future Work

Expand to all **10 classes** in Animals-10

- Train for more epochs with learning rate scheduling

Apply **transfer learning** with MobileNetV2, ResNet50, or EfficientNet

- Tune hyperparameters for higher accuracy

Deploy as a **web app** for live inference